

Viaggio.ai

Developer Workflow Documentation

Complete Guide to Development, Testing, and Deployment

Developer Workflow Overview

Step	Action
1	Create feature branch from main
2	Develop with hot reload (npm run dev)
3	Pre-commit hooks run lint + format on save
4	Push branch → Creates PR
5	CI runs: lint → test → build → E2E
6	Security scan runs: CodeQL + npm audit (FREE)
7	Get code review approval
8	Squash merge to main
9	Auto-deploy to staging
10	Test on staging environment
11	Create release tag for production deploy

Step 1: Create Feature Branch from Main

Ensure you're on main and it's up to date:

```
# Ensure you're on main and it's up to date
git checkout main
git pull origin main

# Create a new branch with proper naming convention
git checkout -b feature/<ticket-id>-<short-description>

# Examples:
git checkout -b feature/VIA-123-hotel-booking
git checkout -b bugfix/VIA-456-search-timeout
git checkout -b hotfix/VIA-789-cors-fix
```

Branch naming conventions:

Prefix	Use Case
feature/	New functionality
bugfix/	Non-urgent bug fixes
hotfix/	Production emergency fixes
docs/	Documentation only
refactor/	Code refactoring

Step 2: Develop with Hot Reload

Backend:

```
cd viaggio-ai-backend

# Install dependencies (first time or after package.json changes)
npm install

# Start development server with hot reload
npm run dev

# Server runs on http://localhost:3001
# Changes to src/ files trigger automatic restart
```

Frontend:

```
cd viaggio-ai-frontend

# Install dependencies
npm install

# Start Vite dev server
npm run dev

# App runs on http://localhost:5173
# Changes trigger instant HMR (Hot Module Replacement)
```

Running both together:

```
# Terminal 1: Backend
cd viaggio-ai-backend && npm run dev

# Terminal 2: Frontend (connects to local backend)
cd viaggio-ai-frontend && VITE_API_URL=http://localhost:3001 npm run dev
```

Step 3: Pre-commit Hooks Run Automatically

When you run git commit, Husky automatically:

- Runs lint-staged on your changed files
- ESLint checks for code issues
- Prettier formats your code

```
# Stage your changes
git add .

# Commit - hooks run automatically
git commit -m "feat: add hotel booking confirmation"

# If hooks fail, fix the issues and commit again
# To see what would be fixed:
npm run lint
npm run format:check
```

Commit message conventions:

feat: add new feature

fix: bug fix

docs: documentation changes

style: formatting, no code change

refactor: code restructuring

test: adding tests

chore: maintenance tasks

Step 4: Push Branch and Create PR

```
# Push your branch to GitHub
git push -u origin feature/VIA-123-hotel-booking

# GitHub will show a link to create a PR
# Or create via GitHub CLI:
gh pr create --title "feat: Add hotel booking confirmation" --body "## Summary
- Added confirmation modal
- Integrated with Hotelbeds API

## Test Plan
- [ ] Manual testing on staging
- [ ] Unit tests pass"
```

PR Description Template:

Summary

Brief description of changes (1-3 bullet points)

Test Plan

- [] How to test this change
- [] Expected behavior

Screenshots (if UI changes)

Step 5: CI Runs Automatically

Once you push, GitHub Actions runs these checks:

Check	Duration	What it does
Lint	~30s	ESLint + Prettier check
Unit Tests	~1-2min	Jest/Vitest tests
Build	~1min	Verify production build works
E2E Tests	~3-5min	Playwright browser tests (frontend)

If CI fails:

```
# Check the GitHub Actions tab for error details
# Fix locally and push again:
```

```
npm run lint:fix      # Auto-fix lint issues
npm run test          # Run tests locally
npm run build         # Verify build works
```

```
git add .
git commit -m "fix: resolve lint errors"
git push
```

Step 6: Security Scan Runs (FREE)

CodeQL Analysis:

- Scans for SQL injection, XSS, command injection
- Results appear in PR checks and Security tab
- Blocks merge if critical vulnerabilities found

npm audit:

- Checks dependencies for known vulnerabilities
- Generates report artifact you can download
- Set to audit-level=high (warns but doesn't block)

If vulnerabilities found:

```
# View vulnerabilities locally
npm audit

# Auto-fix if possible
npm audit fix

# For breaking changes that require major version bumps
npm audit fix --force # Use with caution!
```


Step 7: Get Code Review Approval

For reviewers:

1. Go to PR → Files changed
2. Review each file, leave inline comments
3. Submit review: Approve, Request changes, or Comment

For PR authors:

```
# Address review feedback
git add .
git commit -m "fix: address review feedback"
git push

# Request re-review if needed
gh pr ready
```

Review checklist:

- Code follows project conventions
- Tests cover the changes
- No security concerns
- No performance regressions
- Documentation updated if needed

Step 8: Squash Merge to Main

Once approved and all checks pass:

1. Click 'Squash and merge' button in GitHub
2. Edit the commit message to be descriptive
3. Confirm the merge

Why squash merge:

- Keeps main branch history clean
- One commit per feature/fix
- Easy to revert if needed

```
# After merge, clean up your local branch
git checkout main
git pull origin main
git branch -d feature/VIA-123-hotel-booking
```

Step 9: Auto-Deploy to Staging

What happens automatically:

1. Merge triggers deploy-staging.yml workflow
2. Backend deploys to Railway staging
3. Frontend deploys to Vercel staging
4. Health checks verify deployment success

Staging URLs:

- **Backend:** <https://viaggio-ai-backend-staging.up.railway.app>
- **Frontend:** <https://staging-viaggio.vercel.app>

If deployment fails:

```
# Check GitHub Actions for error logs
# Common issues:
# - Environment variable missing
# - Build error
# - Health check timeout

# Rollback by reverting the merge commit
git revert <merge-commit-sha>
git push origin main
```

Step 10: Test on Staging Environment

Staging Test Checklist:

Core Flows

- Landing page loads
- Search works (tours, hotels)
- Chat with Via responds
- Add to cart works
- Checkout flow completes

New Feature Testing

- Test the specific changes in your PR
- Test edge cases
- Test on mobile viewport

Integration Points

- Viator API responding
- Hotelbeds API responding
- Image identification working

Accessing staging:

```
# Test API directly
curl https://viaggio-ai-backend-staging.up.railway.app/health

# Open frontend in browser
open https://staging-viaggio.vercel.app
```

Step 11: Create Release Tag for Production Deploy

Once staging is verified, promote to production:

Option A: GitHub Release (Recommended)

```
# Create and push a tag
git tag v1.2.3
git push origin v1.2.3

# Or use GitHub CLI
gh release create v1.2.3 --title "v1.2.3" --notes "
## What's New
- Hotel booking confirmation modal
- Improved search performance

## Bug Fixes
- Fixed timeout on large searches
"
```

Option B: Manual Workflow Dispatch

1. Go to Actions → Deploy to Production
2. Click 'Run workflow'
3. Type 'deploy' to confirm
4. Monitor deployment

Version numbering:

- v1.0.0 - Major release (breaking changes)
- v1.1.0 - Minor release (new features)
- v1.1.1 - Patch release (bug fixes)

After production deploy:

```
# Verify production health
curl https://viaggio-ai-backend-production.up.railway.app/health

# Monitor for errors in first 15 minutes
# Check logs in Railway dashboard
```

Quick Reference Commands

Development:

```
npm run dev          # Start dev server
npm run test         # Run tests
npm run test:watch   # Run tests in watch mode
npm run lint         # Check linting
npm run lint:fix     # Auto-fix lint issues
npm run format       # Format code
```

Git workflow:

```
git checkout -b feature/my-feature
git add . && git commit -m "feat: description"
git push -u origin feature/my-feature
gh pr create
```

Testing:

```
npm run test:coverage # Run with coverage report
npm run test:e2e      # Run E2E tests (frontend)
```

Build verification:

```
npm run build        # Production build
npm run preview      # Preview production build (frontend)
```

Important Note: Secrets Cleanup

After implementing the new security workflow, you can remove the `CLAUDE_API_KEY` secret from your GitHub repository settings since the Claude security review will no longer be used.